DISEINU PATROIEN LABORATEGIA

Egileak:

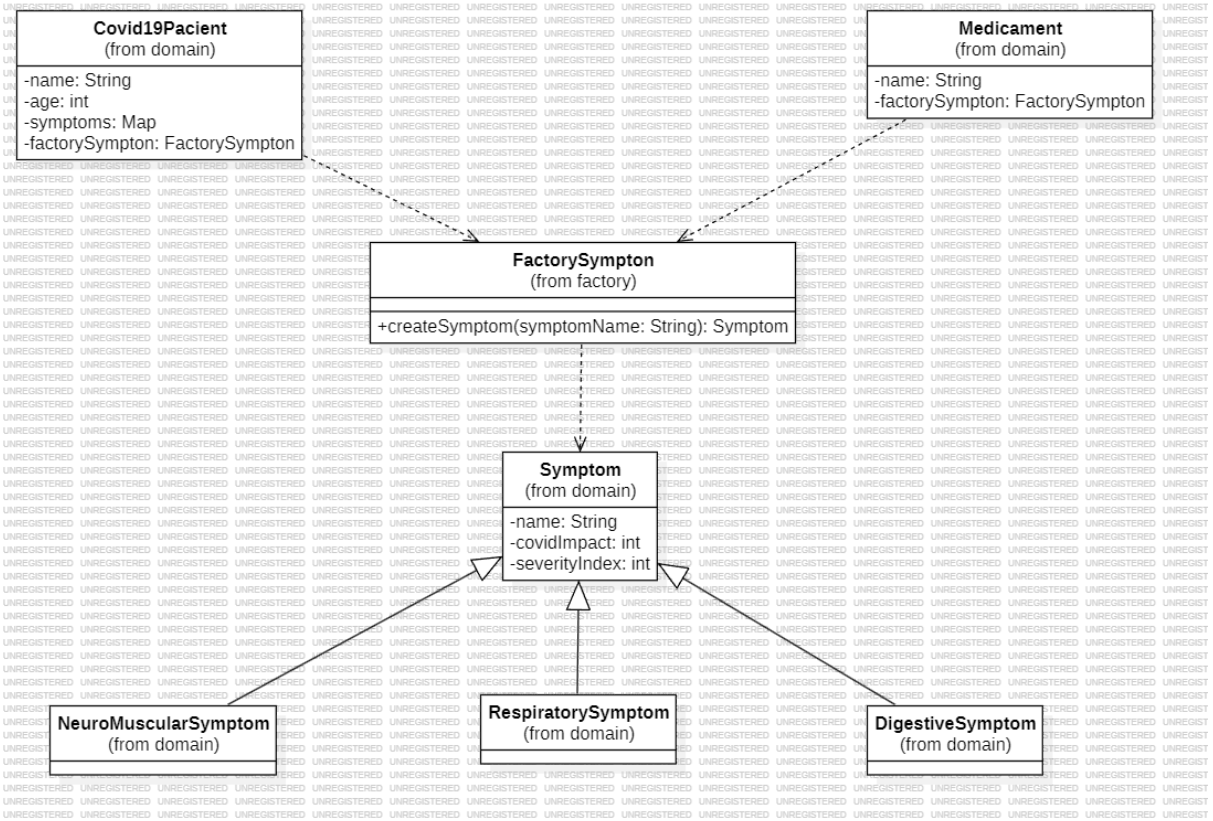
- Danel Larreategui
- Imanol Egaña
- Markel Sanchez

AURKIBIDEA

1. Simple Factory..... 2
2. Observer Patroia..... 4
3. Adapter Patroia.....11
4. Adapter Iterator eta Patroiak.....13
5. Github esteka..... 16

1. Simple Factory

1.1 UML berria eta azalpena:



SymptomFactory klase berria gehitu dugu. Klase honek, sintoma berria sortzearen ardura du eta Covid19Pacient zein Medicament klasearekin erlazioa du. Aldaketa horien medio, etorkizunean sintoma berriak gehitzea askoz errazagoa izango da, bakarrik klase berri hau aldatu beharko delako, beste klase guztiak aldatu gabe.

1.2 Inplementazio berria eta “mareos” sintoma gehitu:

FactorySympton klasea:

```
public class FactorySympton {
        public Symptom createSymptom(String symptomName) {
            List<String> impact5 = Arrays.asList("fiebre", "tos seca", "astenia","expectoracion");
            List<Double> index5 = Arrays.asList(87.9, 67.7, 38.1, 33.4);
            List<String> impact3 = Arrays.asList("disnea", "dolor de garganta", "cefalea","mialgia","escalofrios");
            List<Double> index3 = Arrays.asList(18.6, 13.9, 13.6, 14.8, 11.4);
            List<String> impact1 = Arrays.asList("nauseas", "vómitos", "congestión
nasal","diarrea","hemoptisis","congestion conjuntival", "mareos");
            List<Double> index1 = Arrays.asList(5.0, 4.8, 3.7, 0.9, 0.8, 2.0, 2.0);

            List<String> digestiveSymptom=Arrays.asList("nauseas", "vómitos","diarrea");
            List<String> neuroMuscularSymptom=Arrays.asList("fiebre", "astenia", "cefalea", "mialgia","escalofrios",
"mareos");
            List<String> respiratorySymptom=Arrays.asList("tos seca","expectoracion","disnea","dolor de garganta",
```

```
"congestión nasal","hemoptisis","congestion conjuntival");


            int impact=0;
            double index=0;
            if (impact5.contains(symptomName)) {impact=5; index= index5.get(impact5.indexOf(symptomName));}
              else if (impact3.contains(symptomName)) {impact=3;index=
index3.get(impact3.indexOf(symptomName));}
                 else if (impact1.contains(symptomName)) {impact=1; index=
index1.get(impact1.indexOf(symptomName));}

            if (impact!=0)  {
                  if (digestiveSymptom.contains(symptomName)) return new
DigestiveSymptom(symptomName,(int)index, impact);
                  if (neuroMuscularSymptom.contains(symptomName)) return new
NeuroMuscularSymptom(symptomName,(int)index, impact);
                  if (respiratorySymptom.contains(symptomName)) return new
RespiratorySymptom(symptomName,(int)index, impact);
            }
            return null;

        }
}
```

Main klasea:

```
public class Main {
        public static void main(String[] args) {
                Covid19Pacient p1=new Covid19Pacient("aitor", 35, new FactorySympton());
                //new PacientSymptomGUI(p1);
                new MedicalGUI(new Medicament("Ibuprofeno", new FactorySympton()));
        }
}
```

Medicament klasea:

```
private Symptom createSymptom(String symptomName) {
            return factorySympton.createSymptom(symptomName);
        }
}
```

1.3. Nola egokitu daiteke Factory klasea, Covid19Pacient eta Medicament erabiltzen dituzten Symptom objektuak bakarrak izateko?

createSymptom(String symptomName) metodoari deitzen zaion bakoitzean, lehendabizi HashMap-ean begiratuko genuke eskatutako sintoma jada sortuta dagoen ala ez. Sortuta badago, mapan gordetako instantzia hori bera itzuliko luke. Aldiz, sintoma hori mapan ez badago, objektu berria instanziatu, HashMap-ean gorde eta, itzuli egingo luke.

Modu honetan bermatzen dugu x sintoma sisteman badaude, x Symptom objektu soilik egongo direla, instantzia bakar horiek partekatuko direlako aldiro berriak sortu beharrean.

2. Observer Patroia

2.1. Lehendabiziko prototipoa

2.1.1 Programa nagusia aldatu 2 Covid19Pacient sortzeko bere ondoko PacientSymptomGUI interfazearekin.

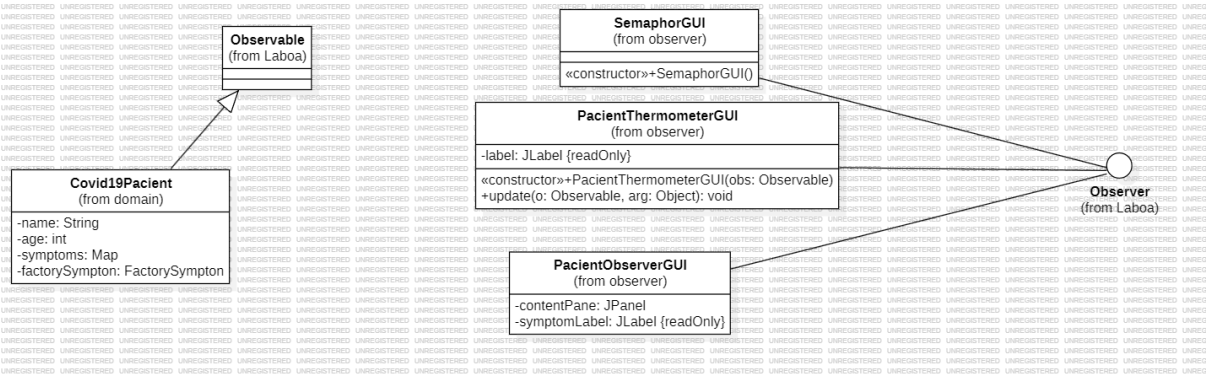
```
public class Main {

	/**
	 * Launch the application.
	 */
	public static void main(String[] args) {
		Covid19Pacient pacient1 = new	Covid19Pacient("aitor",	35, new FactorySympton());
		new	PacientObserverGUI  (pacient1);
		new	PacientSymptomGUI  (pacient1);
		new PacientThermometerGUI (pacient1);
		JFrame gui1 = new ShowPacientTableGUI(pacient1);
		gui1.setPreferredSize(new java.awt.Dimension(300, 200));
		gui1.setVisible(true);

		Covid19Pacient pacient2 = new	Covid19Pacient("aitor",	35, new FactorySympton());
		new	PacientObserverGUI  (pacient2);
		new	PacientSymptomGUI  (pacient2);
		new PacientThermometerGUI (pacient2);
		JFrame gui2 = new ShowPacientTableGUI(pacient2);
		gui2.setPreferredSize(new java.awt.Dimension(300, 200));
		gui2.setVisible(true);
	}
}
```

2.2. Bigarren prototipoa

2.2.1. Aplikazioaren UML diagrama hedatuta egin dituzun aldaketan aurkeztuz.



2.2.2. Aplikazioaren inplementazioa.

PacientObserverGUI

```
public class PacientObserverGUI extends JFrame implements Observer {

	private JPanel contentPane;
	private final JLabel symptomLabel = new JLabel("");
	private Observable obs;

	/**
	 * Create the frame.
	 */
	public PacientObserverGUI(Observable obs) {
		this.obs = obs;
		obs.addObserver(this);
		setTitle("Pacient symptoms");
		setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
		setBounds(650, 100, 200, 300);
		contentPane = new JPanel();
		contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
		setContentPane(contentPane);
		contentPane.setLayout(null);
		symptomLabel.setBounds(19, 38, 389, 199);
		contentPane.add(symptomLabel);
		symptomLabel.setText("Still no symptoms");
		this.setVisible(true);
	}

	@Override
```

```
	public void update(Observable arg0, Object arg1) {
		// TODO Auto-generated method stub
		Covid19Pacient p = (Covid19Pacient) arg0;
		String s = "<html>	Pacient: <b>" + p.getName() + "</b> <br>";
		s = s + "Covid impact: <b>" + p.covidImpact() + "</b><br><br>";
		s = s + " _____  <br>  Symptoms:  <br>";
		Iterator<Symptom> i = p.getSymptoms().iterator();
		Symptom p2;
		while (i.hasNext()) {
			p2 = i.next();
			s = s + "  - " + p2.toString() + ",  " + p.getWeight(p2) + "<br>";
		}
		s = s + "</html>";
		symptomLabel.setText(s);
	}

}
```

PacientSymptomGUI

```
public class PacientSymptomGUI extends JFrame {
	private JPanel contentPane;
	private JTextField weightField;
	JComboBox<Symptom> symptomComboBox;
	private JButton btnRemoveSymptom;
	private JLabel errorLabel;
	private JLabel lblPacient;
	private JLabel labelPacient;


	/**
	 * Create the frame.
	 */
	public PacientSymptomGUI(Covid19Pacient p) {
		setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
		setBounds(200, 100, 450, 300);
		contentPane = new JPanel();
		contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
		setContentPane(contentPane);
		contentPane.setLayout(null);

		JLabel lblSelectSymptom = new JLabel("Select Symptom");
		lblSelectSymptom.setBounds(32, 68, 100, 16);
		contentPane.add(lblSelectSymptom);

		symptomComboBox = new JComboBox<Symptom>();
		symptomComboBox.setBounds(195, 57, 192, 27);
		symptomComboBox.addItem(new Symptom("fiebre",100,5));
		symptomComboBox.addItem(new Symptom("tos seca",100,5));
		symptomComboBox.addItem(new Symptom("astenia",100,5));
		symptomComboBox.addItem(new Symptom("expectoracion",100,5));

		symptomComboBox.addItem(new Symptom("disnea",100,3));
		symptomComboBox.addItem(new Symptom("dolor de garganta",100,3));
```

```
		symptomComboBox.addItem(new Symptom("cefalea",100,3));
		symptomComboBox.addItem(new Symptom("mialgia",100,3));
		symptomComboBox.addItem(new Symptom("escalofríos",100,3));

		symptomComboBox.addItem(new Symptom("náuseas o vómitos",100,1));
		symptomComboBox.addItem(new Symptom("congestión nasal",100,1));
		symptomComboBox.addItem(new Symptom("diarrea",100,1));
		symptomComboBox.addItem(new Symptom("hemoptisis",100,1));
		symptomComboBox.addItem(new Symptom("congestión conjuntival",100,1));


		contentPane.add(symptomComboBox);

		JLabel lblSymptomWeight = new JLabel("Symptom weight");
		lblSymptomWeight.setBounds(32, 96, 125, 16);
		contentPane.add(lblSymptomWeight);

		weightField = new JTextField();
		weightField.setBounds(195, 91, 130, 26);
		contentPane.add(weightField);
		weightField.setColumns(10);

		JButton btnNewButton = new JButton("Add Symptom");
		btnNewButton.addActionListener(new ActionListener() {
			public void actionPerformed(ActionEvent e) {
				errorLabel.setText(" ");
				if (new Integer(weightField.getText())<=3) {
			System.out.println("Symptom added
:"+(Symptom)symptomComboBox.getSelectedItem());


p.addSymptomByName(((Symptom)symptomComboBox.getSelectedItem()).getName(),Integer.parseInt(weightF
ield.getText()));

			} else errorLabel.setText("ERROR, Weight between [1..3]");

			}
		});
		btnNewButton.setBounds(88, 202, 117, 29);
		contentPane.add(btnNewButton);

		btnRemoveSymptom = new JButton("Remove Symptom");
		btnRemoveSymptom.addActionListener(new ActionListener() {
			public void actionPerformed(ActionEvent e) {
				errorLabel.setText(" ");

			System.out.println("Symptom removed
:"+(Symptom)symptomComboBox.getSelectedItem());


p.removeSymptomByName(((Symptom)symptomComboBox.getSelectedItem()).getName());

			}
		});
```

```
                btnRemoveSymptom.setBounds(255, 202, 147, 29);
                contentPane.add(btnRemoveSymptom);

                errorLabel = new JLabel("");
                errorLabel.setBounds(117, 146, 238, 16);
                contentPane.add(errorLabel);

                lblPacient = new JLabel("Pacient:");
                lblPacient.setBounds(210, 17, 61, 16);
                contentPane.add(lblPacient);

            labelPacient = new JLabel("New label");
                labelPacient.setFont(new Font("Lucida Grande", Font.BOLD, 13));
                labelPacient.setBounds(271, 17, 131, 16);
                labelPacient.setText(p.getName());

                contentPane.add(labelPacient);
                this.setVisible(true);
        }
}
```

PacientThermometerGUI

```
public class PacientThermometerGUI extends Frame implements Observer {
        private TemperatureCanvas gauges;
        /**
         * @wbp.nonvisual location=119,71
         */
        private final JLabel label = new JLabel("New label");

        public PacientThermometerGUI(Observable obs) {
                super("Temperature Gauge");
                Panel Top = new Panel();
                add("North", Top);
                gauges = new TemperatureCanvas(0, 15);
                gauges.setSize(500, 280);
                add("Center", gauges);
                setSize(200, 380);
                setLocation(0, 100);
                setVisible(true);
                obs.addObserver(this);
        }

        class TemperatureCanvas extends Canvas {

                public void set(int level) {
                        current = level;
                }

                public int get() {
                        return current;
                }

                public int getMax() {
```

```
                return Max;
        }

        public int getMin() {
                return Min;
        }

        private int Max, Min, current;

        public TemperatureCanvas(int min, int max) {
                Min = min;
                Max = max;
        }

        public void paint(Graphics g) {
                Color c;
                if (current < 5)
                        c = Color.green;
                else if (current < 10)
                        c = Color.yellow;
                else
                        c = Color.red;
                g.setColor(Color.black);
                g.drawRect(left, top, width, height);
                g.setColor(c);
                g.fillOval(left - width / 2, top + height - width / 3, width * 2, width * 2);
                g.setColor(Color.black);
                g.drawOval(left - width / 2, top + height - width / 3, width * 2, width * 2);
                g.setColor(Color.white);
                g.fillRect(left + 1, top + 1, width - 1, height - 1);
                g.setColor(c);
                long redtop = height * (get() - getMax()) / (getMin() - getMax());
                g.fillRect(left + 1, top + (int) redtop, width - 1, height - (int) redtop);
        }

        private static final int width = 20;
        private static final int top = 20;
        private static final int left = 100;
        private static final int right = 250;
        private static final int height = 200;
    }

    @Override
    public void update(Observable o, Object arg) {
        // TODO Auto-generated method stub
        Covid19Pacient p = (Covid19Pacient) o;
//      Obtain the current covidImpact to paint
        int farenheit = (int) p.covidImpact();
//      temperature gauge update
        gauges.set(farenheit);
        gauges.repaint();
    }
}
```

SemaphorGUI

```
public class SemaphorGUI extends JFrame {
        /** stores the associated ConcreteSubject */
        public SemaphorGUI () {
                setSize(100, 100);
                setLocation(350,10);
                Color c=Color.green;
                getContentPane().setBackground(c);
                repaint();
                setVisible(true);
        }
}
```

3. Adapter Patroia

1. eta 2. Eskakizunak:

- Behar den kodea gehitu Covid19PacientTableModelAdapter klasean eta aplikazio exekutatu emaitza konprobatuz.
- Beste paziente bat gehitu sintoma batzuekin, eta aplikazioa exekutatu bi taulak agertzen direla konprobatuz.

```
pacient.addSymptomByName("disnea", 2);
pacient.addSymptomByName("cefalea", 1);
pacient.addSymptomByName("astenia", 3);

pacient2.addSymptomByName("cefalea", 2);
pacient2.addSymptomByName("tos seca", 1);
pacient2.addSymptomByName("astenia", 3);
```

Symptom	Weight
disnea	2
astenia	3
cefalea	1

Symptom	Weight
tos seca	1
cefalea	2
astenia	3

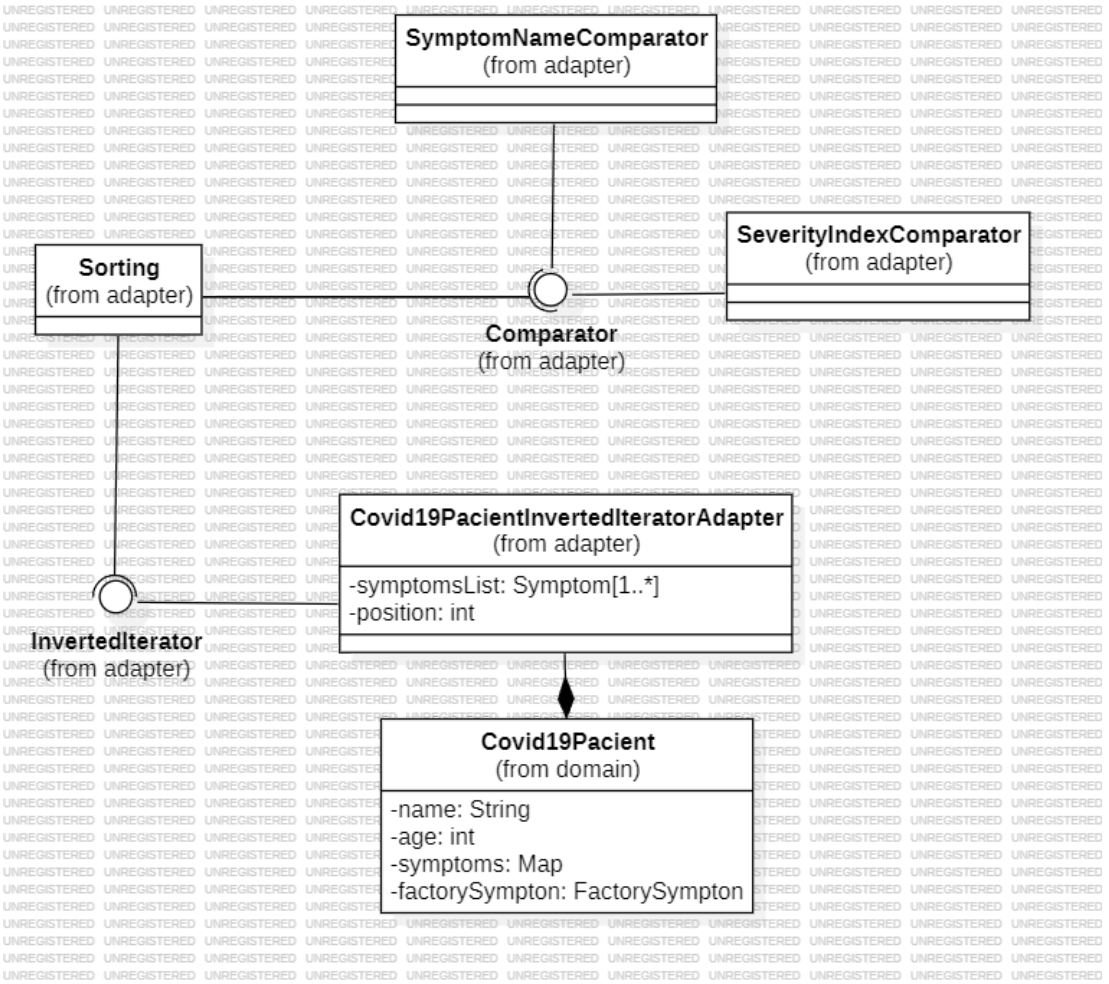
3. Eskakizuna:

- Nola gehituko zenuke taula lehio bat (ShowPacientTableGUI) aurreko observer ariketan, paziente bateri sintoma berri bat gehitzen zaion bakoitzean bere taula leihoa eguneratzeko?

```
public class Covid19PacientTableModelAdapter extends AbstractTableModel {
        protected Covid19Pacient pacient;
        protected String[] columnNames =
          new String[] {"Symptom", "Weight" };
        protected Vector<Symptom> symptoms;
        public Covid19PacientTableModelAdapter(Covid19Pacient p) {
          this.pacient=p;
          symptoms = new Vector<>(p.getSymptoms());
        }
        public int getColumnCount() {
          // Challenge!
                return 2;
        }
        public String getColumnName(int i) {
          // Challenge!
                 return columnNames[i];
        }
        public int getRowCount() {
          // Challenge!
                 return pacient.getSymptoms().size();
        }
        public Object getValueAt(int row, int col) {
                symptoms = new Vector<>(pacient.getSymptoms());
          // Challenge!
                 if(col == 0) {
                         return symptoms.get(row).getName();
                 }
                 if(col == 1) {
                         return pacient.getWeight(symptoms.get(row));
                 }
                 return null;
        }
}
```

4. Adapter Iterator eta Patroiak

1. Pausoa: UML:



2. Pausoa:

Comparator interfazea inplementatzen dituzten bi klase definitu elementuak symptomName eta severityIndex ordenatzen dituztenak hurrenez hurren.

```
public class SymptomNameComparator implements Comparator<Object> {
    @Override
    public int compare(Object o1, Object o2) {
        Symptom s1 = (Symptom) o1; // Cast a Symptom
        Symptom s2 = (Symptom) o2;
        return s1.getName().compareToIgnoreCase(s2.getName());
    }
}
```

```
public class SeverityIndexComparator implements Comparator<Object> {
 @Override
 public int compare(Object o1, Object o2) {
    Symptom s1 = (Symptom) o1;
    Symptom s2 = (Symptom) o2;
    return Integer.compare(s1.getSeverityIndex(), s2.getSeverityIndex());
 }
}
```

3. Pausoa:

Covid19Pacient klasea InvertedIterator interfazera egokitzen duen klase adaptadorea sortu. Gogoratu metodo eraikitzaile egokia sortzea pazientearen informazioa bidaltzeko.

```
public class Covid19PacientInvertedIteratorAdapter implements InvertedIterator {
   private List<Symptom> symptomsList;
   private int position;

   // Metodo eraikitzailea pazientearen informazioa jasotzeko
   public Covid19PacientInvertedIteratorAdapter(Covid19Pacient pacient) {
      this.symptomsList = new ArrayList<>(pacient.getSymptoms());
      this.position = symptomsList.size() - 1;
   }

   @Override
   public Object previous() {
      Symptom symptom = symptomsList.get(position);
      position--;
      return symptom;
   }

   @Override
   public boolean hasPrevious() {
      return position >= 0;
   }

   @Override
   public void goLast() {
      this.position = symptomsList.size() - 1;
   }
}
```

4. Pausoa:

Programa nagusi batean, Covid19Pacient objektu bat sortu 5 sintomekin, eta Sorting.sort metodoari bi aldiz deitu, sortu duzun CovidPacient klase adaptadorea eta Comparator inplementatu dituzun bi konparadorearekin. Bukatzeko emaitza inprimatu pantaiatik.

```
public class MainAdapterIterator {
    public static void main(String[] args) {
        // 1. Pazientea sortu 5 sintomarekin
        Covid19Pacient pacient = new Covid19Pacient("Ane", 29, new FactorySympton());
        pacient.addSymptomByName("fiebre", 2);
        pacient.addSymptomByName("tos seca", 1);
        pacient.addSymptomByName("astenia", 3);
        pacient.addSymptomByName("disnea", 2);
        pacient.addSymptomByName("cefalea", 1);

        // 2. Egokitzailea (Adapter) sortu pazientearen datuekin
        Covid19PacientInvertedIteratorAdapter adapter = new Covid19PacientInvertedIteratorAdapter(pacient);

        // 3. Izenaren arabera ordenatu eta inprimatu
        System.out.println("--- Sintomak IZENAREN arabera ordenatuta ---");
        Iterator<Object> itByName = Sorting.sortedIterator(adapter, new SymptomNameComparator());
        while(itByName.hasNext()) {
            System.out.println(itByName.next());
        }

        // 4. Larritasun indizearen arabera ordenatu eta inprimatu
        System.out.println("\n--- Sintomak LARRITASUN INDIZEAREN (severityIndex) arabera ordenatuta ---");
        Iterator<Object> itBySeverity = Sorting.sortedIterator(adapter, new SeverityIndexComparator());
        while(itBySeverity.hasNext()) {
            System.out.println(itBySeverity.next());
        }
    }
}
```

5. Github esteka

https://github.com/Imanol-16/labpatterns.git